

Cahier de TP

« Administration Cluster Kafka »

Pré-requis :

- Bonne connexion Internet
- Linux
- JDK21+, Maven
- IDE Recommandés : STS 4, IntelliJIDEA, VSCode
- Docker, Git

Solutions :

<https://github.com/dthibau/kafka-admin-solutions>

Images docker utilisées :

- *apache/kafka:4.2.0*
- [docker.redpanda.com/redpandadata/console:v3.2.1](#)
- [tchiotludo/akhq](#)
- [digitsy/kafka-magic](#)
- [confluentinc/cp-kafka-rest:7.7.8](#)
- [confluentinc/cp-schema-registry:7.7.8](#)
- *postgres:15.1*
- *dpage/pgadmin4*
- [docker.elastic.co/elasticsearch/elasticsearch:8.13.0](#)
- [docker.elastic.co/kibana/kibana:8.13.0](#)
- [quay.io/keycloak/keycloak:26.1](#)

Table des matières

Atelier 1: Mise en place.....	4
1.1 Installation cluster.....	4
1.1.1 Option archive.....	4
1.1.2 Option Docker.....	5
1.1.3 Vérifications.....	5
1.2 Utilitaires : Configuration dynamique et inspection du cluster.....	6
1.2.1 Inspection du quorum KRaft.....	6
1.2.2 Modifier la configuration d'un topic.....	6
1.2.3 Configuration dynamique du broker.....	7
1.3 Outils graphiques.....	7
Atelier 2: Production de messages.....	8
Atelier 3 : Consommation de messages.....	9
3.0 Mise en place BD.....	9
3.1 Performance.....	9
3.2 Rebalancing des consommateurs.....	10
Atelier 4 : Exactly Once et Transactions.....	11
4.1 Transaction et ISOLATION_LEVEL.....	11
4.1.1 Producteur transactionnel.....	11
4.1.2 Consommateur.....	11
4.2 Exactly Once.....	11
Atelier 5. Schema Registry et sérialisation Avro.....	12
5.1 Démarrage de Confluent Registry.....	12

5.2 Producteur de message.....	12
5.3 Consommateur de message.....	12
5.4 Evolution du schéma compatible.....	13
5.5 Evolution du schéma incompatible.....	13
Atelier 6. Kafka Connect.....	14
6.1 Installation du plugin ElasticSearch dans kafka connect.....	14
6.2 Démarrage de la stack.....	14
6.3 Configuration du connecteur.....	14
6.4 Améliorations.....	15
Atelier 7. MirrorMaker 2.....	16
7.1 Démarrer le second cluster.....	16
7.2 Vérifier la disponibilité des connecteurs MM2.....	17
7.3 Créer le connecteur MirrorSource.....	17
7.4 Créer le connecteur MirrorCheckpoint.....	18
7.5 Créer le connecteur MirrorHeartbeat.....	18
7.6 Vérifier la réplication.....	18
Atelier 8 : kafkaStream et ksqlDB.....	20
8.1 Application KafkaStream.....	20
8.2 kSQLDb.....	20
8.2.1 Démarrage de la stack.....	20
8.2.1 Getting Started.....	20
8.2.2 Transformations et filtres.....	21
8.2.3 Requêtes.....	22
8.2.4 Pour aller plus loin.....	23
Atelier 9: Sécurité.....	24
9.1 Séparation des échanges réseaux.....	24
9.2 Mise en place de SSL pour crypter les données.....	24
9.2.1 SSL appliqué aux communications inter-broker.....	24
9.2.2 SSL appliqué aux communications externes.....	25
9.3 Authentification via SASL/PLAIN.....	25
9.3.1 Authentification inter-broker.....	25
9.3.2 Authentification client.....	26
9.4 ACL.....	27
9.4.1 Activation des ACLs.....	27
9.4.2 Définition ACLs.....	27
9.5 Quota.....	27
Atelier 10 : Exploitation.....	29
10.1 Rétention et segments.....	29
10.2 Extension du cluster et réassignement des partitions.....	29
10.3 Rolling restart.....	29
10.4 Bascule et reprise depuis le cluster B.....	29
10.4.1. Préparer un consumer group côté A.....	29
10.4.2. Inspecter les checkpoints côté B.....	30
10.4.3. Simuler la panne de A.....	30
10.4.4. Faire reprendre le consumer côté B.....	30
10.4.5. Mesurer le RPO.....	30
10.4.6. Failback.....	31
Atelier 11. <i>Monitoring avec Prometheus/Grafana</i>	32
11.1 Exporter les informations JMX au format Prometheus.....	32
9.3 Mise en place Prometheus Grafana.....	32
A. Annexes	34
A.1. Installation Ensemble Zookeeper.....	34

Atelier 1: Mise en place

1.1 Installation cluster

1.1.1 Option archive

Étape 1 : Création du cluster ID et formatage des répertoires de logs

Télécharger et dézipper la distribution de Kafka dans un répertoire ***\$KAFKA_HOME***

Créer un répertoire ***\$KAFKA_LOGS*** qui stockera les messages des 3 brokers

Créer un répertoire ***\$KAFKA_CONFIG***

Copier le fichier de la distribution ***\$KAFKA_HOME/config/server.properties*** dans 3 fichiers ***server1.properties***, ***server2.properties***, ***server3.properties*** placés dans le répertoire ***\$KAFKA_CONFIG***

Éditer les 3 fichier ***serverX.properties*** afin de modifier les propriétés :

- ***node.id***
- ***listeners (utiliser les ports 9092,9192 et 9292 ET 9093,9193 et 9293)***
- ***advertised.listeners***
- ***controller.quorum.voters***
- ***log.dir*** à ***\$KAFKA_LOGS/broker-<id>***
- ***default.replication.factor : 3***

Et d'autres propriétés de topic qui vous paraissent intéressantes

Générer un id de cluster :

\$KAFKA_HOME/bin/kafka-storage.sh random-uuid

Utiliser l'ID du cluster pour formater les 3 répertoires

Pour chaque broker, formater son répertoire de log avec :

\$KAFKA_HOME/bin/kafka-storage.sh format -t <cluster_id> -c \$KAFKA_CONFIG/serverX.properties

Étape 2 : Mise au point d'un script de démarrage/arrêt

Mettre au point un script permettant de démarrer les 3 brokers en mode daemon :

Exemple :

```
#!/bin/sh
```

```
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64/  
export KAFKA_HOME=<répertoire de la distribution>
```

```
$KAFKA_HOME/bin/kafka-server-start.sh -daemon config/server1.properties
$KAFKA_DIST/bin/kafka-server-start.sh -daemon config/server2.properties
$KAFKA_DIST/bin/kafka-server-start.sh -daemon config/server3.properties
```

Visualiser les traces de démarrages :

```
tail -f $KAFKA_DIST/logs/server.log
```

Mettre au point un script d'arrêt

```
#!/bin/sh
```

```
export KAFKA_HOME=<répertoire de la distribution>
```

```
$KAFKA_HOME/kafka-server-stop.sh
```

Effectuer les vérifications de création de topic et envoi/réception de messages

```
export PATH=$KAFKA_DIST/bin:$PATH
```

```
kafka-topics.sh --create --bootstrap-server localhost:9092 --replication-factor 1 --partitions 1 --
topic test
```

```
kafka-console-producer.sh --bootstrap-server localhost:9092 --topic test
```

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test --from-beginning
```

1.1.2 Option Docker

Visualiser le fichier ***docker-compose.yml***

Démarrer la stack et observer les logs de démarrages

1.1.3 Vérifications

Si docker :

```
docker exec -it kafka-0 bash
```

KAFKA_HOME est alors dans /opt/kafka/bin

Créer un topic ***testing*** avec 5 partitions et 2 répliques :

```
$KAFKA_HOME/bin/kafka-topics.sh --create --bootstrap-server localhost:9092 --
replication-factor 2 --partitions 5 --topic testing
```

Lister les topics du cluster

Accéder à la description détaillée du topic

Démarrer un producteur de message

```
$KAFKA_HOME/bin/kafka-console-producer.sh -- bootstrap-server localhost:9092 --
```

```
topic testing --property "parse.key=true" --property "key.separator=:"
```

Saisir quelques messages

Consommer les messages depuis le début

```
$KAFKA_HOME/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --  
topic testing --from-beginning
```

Dans une autre fenêtre, lister les groupes de consommateurs et accéder au détail du groupe de consommateur en lecture sur le topic *testing*

1.2 Utilitaires : Configuration dynamique et inspection du cluster

Objectifs

Utiliser `kafka-configs.sh` pour modifier la configuration sans redémarrer le cluster, et `kafka-metadata-quorum.sh` pour inspecter l'état du quorum KRaft.

1.2.1 Inspection du quorum KRaft

Afficher l'identifiant du cluster :

```
bin/kafka-cluster.sh cluster-id --bootstrap-server localhost:9092
```

Afficher le statut du quorum de controllers :

```
bin/kafka-metadata-quorum.sh --bootstrap-server localhost:9092 \  
describe --status
```

Questions :

- Quel est le nœud leader du quorum ?
- Quel est le LeaderEpoch actuel ?
- Que signifie le champ HighWatermark ?

Afficher la réplication entre controllers :

```
bin/kafka-metadata-quorum.sh --bootstrap-server localhost:9092 \  
describe --replication
```

Question : quel est le Lag de chaque follower ? Que se passerait-il s'il devenait élevé ?

1.2.2 Modifier la configuration d'un topic

Afficher la configuration actuelle du topic `testing` :

```
bin/kafka-configs.sh --bootstrap-server localhost:9092 \  
--entity-type topics --entity-name my-topic \  
--describe
```

Question : la sortie est-elle vide ou contient-elle des paramètres ? Pourquoi ?

Réduire la rétention à 1 heure (3 600 000 ms) sur ce topic uniquement :

```
bin/kafka-configs.sh --bootstrap-server localhost:9092 \  
--entity-type topics --entity-name my-topic \  
--alter --add-config retention.ms=3600000
```

Vérifier avec `--describe` que la modification est prise en compte.

Annuler cette surcharge (revenir à la valeur du broker) :

```
bin/kafka-configs.sh --bootstrap-server localhost:9092 \  
  --entity-type topics --entity-name my-topic \  
  --alter --delete-config retention.ms
```

Question : pourquoi est-il préférable de supprimer une surcharge plutôt que de la remettre à la valeur par défaut « manuellement » ?

1.2.3 Configuration dynamique du broker

Afficher les configurations dynamiques d'un broker :

```
bin/kafka-configs.sh --bootstrap-server localhost:9092 \  
  --entity-type brokers --entity-name 1 \  
  --describe
```

Modifier dynamiquement le niveau de log d'une classe Kafka, sans redémarrage :

```
bin/kafka-configs.sh --bootstrap-server localhost:9092 \  
  --entity-type broker-loggers --entity-name 1 \  
  --alter --add-config kafka.controller.KafkaController=DEBUG
```

Provoquer une activité (créer un topic, par exemple) puis observer les logs du broker. Les nouveaux messages DEBUG apparaissent.

Revenir au niveau de log précédent :

```
bin/kafka-configs.sh --bootstrap-server localhost:9092 \  
  --entity-type broker-loggers --entity-name 1 \  
  --alter --delete-config kafka.controller.KafkaController
```

1.3 Outils graphiques

Plusieurs outils graphique existent, un docker compose est fourni pour démarrer *akhq*, *kafka-magic* et *redpanda-console*

```
cd $TP_Admin/1.3_UI
```

```
docker compose -f kafka-ui-docker-compose.yml up -d
```

Atelier 2: Production de messages

Le projet fourni est un projet développé avec le framework Spring.

Il modélise une flotte de 100 coursiers qui envoient 10000 fois leur position (latitude/longitude) toutes les 10 millisecondes. Les messages sont numérotés par Coursier

Lors du démarrage du programme, le producteur affiche sa configuration sur la console et démarre la production de message après que l'on ait appuyé sur la touche Enter.

Le producteur peut être configuré via la ligne de commande par la propriété **async** activant ou désactivant le mode asynchrone d'envoi et en préfixant les propriétés Kafka par *spring.kafka.producer*

Exemple :

```
java -jar SpringProducer.jar --spring.kafka.producer.acks=0 --async=true
```

Créer le topic **position** avec 5 partitions et le facteur de réplication de 3 et un *min.insync.replica* de 2 :

```
./kafka-topics.sh --create --bootstrap-server localhost:9092 --partitions 5 --replication-factor 3 --config min.insync.replicas=2 --topic position
```

Faire plusieurs essais de production de message en réinitialisant le topic et en changeant les paramètres par défaut, Vérifier la configuration à chaque fois

- **acks**
Pour comparer les valeurs de débits
- **batch.size** et **linger.ms**
Pour comparer les valeurs de débits
- **retries / max.in.flights.requests.per.session**
Avec arrêt/redémarrage du cluster ou d'un serveur durant les tests.
Vérifier le nombre de messages produits

Atelier 3 : Consommation de messages

Le projet fourni est également un projet développé avec le framework Spring.

Il permet de consommer le topic *position*.

Le nom du groupe est *consumer*

Il peut être surchargé via la ligne de commande :

--spring.kafka.consumer.group-id

La propriété de configuration *spring.kafka.listener.concurrency* fixe le nombre de consommateurs du groupe.

Elle est fixée à 3 et peut être surchargée via la ligne de commande.

--spring.kafka.listener.concurrency

Le programme consomme les messages et stocke dans une base postgres l'id du coursier et l'id de l'index du message:

- En permanence, il affiche des informations sur le débit de traitement
- Si la base est réinitialisée, il détecte les doublons de traitements

3.0 Mise en place BD

```
docker compose -f postgres-docker-compose.yml up -d
```

Accéder à *localhost:81* et se logger avec *admin@admin.com/admin*

Enregistre un serveur avec comme paramètres de connexion :

- host : *consumer-postgresql*
- user : *postgres*
- password : *postgres*

3.1 Performance

Démarrer le programme avec les valeurs de concurrence par défaut :

java -jar SpringConsumer.jar

Visualiser les logs de démarrage de l'application qui affichent :

- La configuration du consommateur
- Les allocations de partition

Supprimer les informations du groupe dans Kafka afin que les consommateurs repartent à zéro.

Modifier des paramètres pouvant affecter la performance et les commits d'offset :

- *max.poll.records*

3.2 Rebalancing des consommateurs

Démarrer 3 processus avec un degré de concurrence à 1

java -jar SpringConsumer.jar --spring.kafka.listener.concurrency=1

Arrêter un processus et observer la réaffectation de partition

Redémarrer le processus

Atelier 4 : Exactly Once et Transactions

Objectifs :

- Visualiser des messages committés et non-committés
- Voir les effets de la propriété *ISOLATION_LEVEL*
- Implémenter un consommateur *ExactlyOnce*

4.1 Transaction et ISOLATION_LEVEL

4.1.1 Producteur transactionnel

Le producteur fournit englobe dans une transaction des batchs de messages de taille 10.

Lancer le programme avec :

```
java -jar producer-tx.jar 1 105 100 0
```

Ce qui signifie que 1 threads envoie 105 messages, les 100 premiers messages font partie de 10 transactions validées. Les 5 derniers messages ne sont pas validés.

Visualiser les messages dans une console d'administration.

Noter le nombre et les offsets, Visualiser le flag transactionnel

4.1.2 Consommateur

Le programme consommateur peut être démarré en mode *READ_COMMITTED* ou *READ_UNCOMMITTED*

Faire un premier démarrage en mode *READ_UNCOMMITTED*

```
java -jar consumer.jar 1 false position-tx
```

Noter le nombre de messages consommés

Supprimer le groupe de consommateur et démarrer le message en mode *READ_COMMITTED*

```
java -jar consumer.jar 1 true position-tx
```

Noter le nombre de messages consommés

4.2 Exactly Once

Reprendre le nouveau consommateur.

Ce nouveau programme lit le topic *position-tx*, ajoute une information de distance à l'enregistrement et publie un message vers le topic *position_distance*.

Exécuter le programme :

```
java -jar consumer-exactly-once.jar 1
```

Tuer le processus pendant la consommation puis relancer le programme

Vérifier les messages produits en utilisant le programme précédent en READ_COMMITTED

```
java -jar consumer.jar 1 true position_distance
```

Atelier 5. Schema Registry et sérialisation Avro

5.1 Démarrage de Confluent Registry

Visualiser le fichier ***docker-compose.yml*** fourni qui démarre Redpanda Console, akhq et SchemaRegistry de confluent

Démarrer la stack via :

```
docker compose up -d
```

Vérifier le bon démarrage des containers

Accéder à *localhost:8085/subjects*

5.2 Producteur de message

Récupérer le projet ***producer-avro***

Visualiser le schéma Avro :

src/main/resources/Courier.avsc

Lancer la commande ***./mvnw compile*** et regarder les classes générées par le plugin Avro

Lancer ensuite ***./mvnw package*** pour construire l'exécutable

Démarrer l'exécutable par

```
java -jar target/producer-avro-0.0.1-SNAPSHOT-jar-with-dependencies.jar 10 500 10 0
```

Le programme, après avoir enregistré le schéma, envoie 10*500 messages qui sont sérialisés avec Avro sur le topic ***avro-position***

Après un premier démarrage, le schéma doit être consultable :

- sur schema-registry (localhost:8085/subjects)
- ou via votre outil d'admin

Vérifier que l'outil graphique puisse lire les messages Avro

5.3 Consommateur de message

Récupérer le projet ***consumer-avro***

Visualiser le code source et en particulier la boucle de poll et l'utilisation de ***GenericMessage***

Lancer ensuite ***./mvnw package*** pour construire l'exécutable

Consommer les messages du topic précédent avec la commande

```
java -jar target/consumer-0.0.1-SNAPSHOT-jar-with-dependencies.jar 2 1
```

Laisser le programme tourner

5.4 Evolution du schéma compatible

Mettre à jour le schéma en ajoutant les champs optionnels : **firstName** dans la structure **Coursier**

```
{
    "name": "first_name",
    "type": "string",
    "default": "undefined"
},
```

Fixer les problèmes de compilation :

Dans **src/main/java/org/formation/KafkaProducerThread.java**

Changer la ligne 29 par :

```
this.courier = new Courier(id, "David", new Position(Math.random() + 45,
Math.random() + 2));
```

Reconstruire l'application via :

./mvnw clean package

Relancer le programme de production

java -jar target/producer-avro-0.0.1-SNAPSHOT-jar-with-dependencies.jar 10 500 10 0

Visualiser ensuite la nouvelle version du schéma dans le registre

Consommer les messages sans modifications du programme consommateur

5.5 Evolution du schéma incompatible

Mettre à jour le schéma en ajoutant un champs obligatoire : **vehicle_id** dans la structure **Coursier**

```
{
    "name": "vehicle_id",
    "type": "int"
},
```

Fixer les problèmes de compilation

Dans **src/main/java/org/formation/KafkaProducerThread.java**

Changer la ligne 29 par :

```
this.courier = new Courier(id, "David", 1, new Position(Math.random() + 45,
Math.random() + 2));
```

Relancer le programme de production et visualiser l'exception au moment de l'enregistrement du nouveau schéma.

Atelier 6. Kafka Connect

Objectif :

- Déverser le topic position dans un index ElasticSearch

6.1 Installation du plugin ElasticSearch dans kafka connect

Visualiser le fichier Dockerfile fourni et construire une image *my-kafka-connect-with-elasticsearch* via :

```
docker build -t my-kafka-connect-with-elasticsearch .
```

6.2 Démarrage de la stack

Démarrer la nouvelle stack qui ajoute l'image précédemment construite, ElasticSearch et Kibana
docker-compose up -d

Vérifier l'installation du plugin via :

<http://localhost:8083/connector-plugins>

Se connecter à kibana localhost:5601 et dans la DevConsole exécuter :

```
PUT /position
{
  "mappings":{
    "properties": {
      "@timestamp": {
        "type": "date",
        "format": "epoch_millis"
      }
    }
  }
}
```

La commande crée un index elasticsearch *position* avec pour l'instant un seul champ *@timestamp*

6.3 Configuration du connecteur

Définir un connecteur *elasticsearch-sink* qui déverse le topic position dans l'index position :

```
curl -X POST http://localhost:8083/connectors \
-H "Content-Type: application/json" \
-d '{
  "name": "elasticsearch-sink",
  "config": {
    "connector.class":
"io.confluent.connect.elasticsearch.ElasticsearchSinkConnector",
    "tasks.max": "1",
    "topics": "position",
    "topic.index.map": "position:position_index",
    "connection.url": "http://elasticsearch:9200",
    "type.name": "log",
    "key.ignore": "true",
    "schema.ignore": "true",
    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
    "key.converter.schemas.enable": "false",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter.schemas.enable": "false"
  }
}
```

Vous pouvez vérifier la bonne installation du connecteur :

- Via les logs de KafkaConnect
- Via `http://localhost:8083/connectors` : Liste des connecteurs actifs
- Via l'interface de RedPanda

Alimenter le topic position

Vous pouvez visualiser les effets du connecteur

- Via ElasticSearch : `http://localhost:9200/position/_search`
- Via Kibana : <http://localhost:5601>

6.4 Améliorations

- Améliorer le fichier de configuration afin d'introduire que le timestamp Kafka soit renseigné dans le champ `@timestamp` de l'index position d'ElasticSearch
- Déverser le topic ***position-avro*** dans un index ***position-avro***

Atelier 7. MirrorMaker 2

Objectifs :

- Répliquer le topic position du cluster Kafka existant vers un second cluster mono-broker en utilisant les connecteurs MirrorMaker 2

7.1 Démarrer le second cluster

On ajoute un cluster cible kafka-b au déploiement existant via un fichier d'override docker-compose.mm2.yml.

Ce cluster est mono-broker pour rester simple ; il rejoint le réseau kafka-net afin d'être joignable depuis le service kafka-connect.

Démarrer le second cluster en complément de la stack existante :

```
docker-compose -f docker-compose.yml -f docker-compose.mm2.yml up -d kafka-b
```

Vérifier que le broker est up :

```
docker exec -it kafka-b /opt/kafka/bin/kafka-broker-api-versions.sh \
--bootstrap-server localhost:29092
```

7.2 Vérifier la disponibilité des connecteurs MM2

Les trois connecteurs MirrorMaker 2 sont fournis avec Apache Kafka. Lister les plugins disponibles dans Kafka Connect :

```
curl -s http://localhost:8083/connector-plugins | jq '[] | select(.class | contains("Mirror"))'
```

On doit voir les trois classes :

- org.apache.kafka.connect.mirror.MirrorSourceConnector
- org.apache.kafka.connect.mirror.MirrorCheckpointConnector
- org.apache.kafka.connect.mirror.MirrorHeartbeatConnector

7.3 Créer le connecteur MirrorSource

Ce connecteur réplique le topic position du cluster A vers le cluster B.

Le topic répliqué sera nommé A.position côté B (préfixé par alias source, défaut de DefaultReplicationPolicy).

```
curl -X POST http://localhost:8083/connectors \
-H "Content-Type: application/json" \
-d '{
  "name": "mm2-source-A-to-B",
  "config": {
    "connector.class": "org.apache.kafka.connect.mirror.MirrorSourceConnector",
    "tasks.max": "1",
    "source.cluster.alias": "A",
    "target.cluster.alias": "B",
    "source.cluster.bootstrap.servers": "localhost:9092,localhost:9192,localhost:9292",
    "target.cluster.bootstrap.servers": "localhost:29092",
    "producer.override.bootstrap.servers": "localhost:29092",
    "topics": "position",
    "replication.factor": "1",
    "offset-syncs.topic.replication.factor": "1",
```

```
"key.converter": "org.apache.kafka.connect.converters.ByteArrayConverter",
"value.converter": "org.apache.kafka.connect.converters.ByteArrayConverter"
}
}'
```

Note : *ByteArrayConverter* est utilisé pour préserver le payload tel quel pendant la réplication, indépendamment du format métier.

Vérifier avec :

```
docker exec -it kafka-b /opt/kafka/bin/kafka-topics.sh --bootstrap-server
localhost:29092 --list
```

Puis consommer avec :

```
docker exec -it kafka-b /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server
localhost:29092 --topic A.position --from-beginning
```

7.4 Créer le connecteur *MirrorCheckpoint*

Ce connecteur synchronise les offsets de consumer groups du cluster A vers le cluster B (utile pour le failover, mais pas indispensable pour la démo de réplication seule).

```
curl -X POST http://localhost:8083/connectors \
-H "Content-Type: application/json" \
-d '{
  "name": "mm2-checkpoint-A-to-B",
  "config": {
    "connector.class": "org.apache.kafka.connect.mirror.MirrorCheckpointConnector",
    "tasks.max": "1",
    "source.cluster.alias": "A",
    "target.cluster.alias": "B",
    "source.cluster.bootstrap.servers": "localhost:9092,localhost:9192,localhost:9292",
    "target.cluster.bootstrap.servers": "localhost:29092",
    "producer.override.bootstrap.servers": "localhost:29092",
    "checkpoints.topic.replication.factor": "1",
    "emit.checkpoints.interval.seconds": "5"
  }
}'
```

7.5 Créer le connecteur *MirrorHeartbeat*

Ce connecteur émet un heartbeat périodique sur le cluster source ; le *MirrorSourceConnector* répliquera ces heartbeats côté cible.

Cela permet de mesurer la latence et la santé du lien.

```
curl -X POST http://localhost:8083/connectors \
-H "Content-Type: application/json" \
-d '{
  "name": "mm2-heartbeat-A",
  "config": {
    "connector.class": "org.apache.kafka.connect.mirror.MirrorHeartbeatConnector",
    "tasks.max": "1",
    "source.cluster.alias": "A",
    "target.cluster.alias": "B",
    "source.cluster.bootstrap.servers": "localhost:9092,localhost:9192,localhost:9292",
```

```

    "target.cluster.bootstrap.servers": "localhost:29092",
"producer.override.bootstrap.servers": "localhost:9092,localhost:9192,localhost:9292",
    "heartbeats.topic.replication.factor": "1",
    "emit.heartbeats.interval.seconds": "5"
  }
}'

```

Mettre à jour le connecteur afin qu'il réplique le topic heartbeats

```

curl -X PUT http://localhost:8083/connectors/mm2-source-A-to-B/config \
-H "Content-Type: application/json" \
-d '{
  "connector.class": "org.apache.kafka.connect.mirror.MirrorSourceConnector",
  "tasks.max": "1",
  "source.cluster.alias": "A",
  "target.cluster.alias": "B",
  "source.cluster.bootstrap.servers": "localhost:9092,localhost:9192,localhost:9292",
  "target.cluster.bootstrap.servers": "localhost:29092",
  "producer.override.bootstrap.servers": "localhost:29092",
  "topics": "position|heartbeats",
  "replication.factor": "1",
  "offset-syncs.topic.replication.factor": "1",
  "key.converter": "org.apache.kafka.connect.converters.ByteArrayConverter",
  "value.converter": "org.apache.kafka.connect.converters.ByteArrayConverter"
}'

```

Vérifier l'état des trois connecteurs :

```
curl -s "http://localhost:8083/connectors?expand=status" | jq
```

7.6 Vérifier la réplication

Lister les topics côté cluster B — on doit y trouver __consumer_offsets,A.position, A.heartbeats, heartbeats (pour le fail-over) et le topic interne mm2-offset-syncs.B.internal :

```

docker exec -it kafka-b /opt/kafka/bin/kafka-topics.sh \
--bootstrap-server kafka-b:9092 --list

```

Produire des messages sur position côté A :

```

docker exec -it kafka-0 /opt/kafka/bin/kafka-console-producer.sh \
--bootstrap-server kafka-0:9092 --topic position

```

Et lire en parallèle sur A.position côté B pour observer la réplication en quasi temps réel :

```

docker exec -it kafka-b /opt/kafka/bin/kafka-console-consumer.sh \
--bootstrap-server kafka-b:9092 --topic A.position --from-beginning.

```

Atelier 8 : kafkaStream et ksqlDB

8.1 Application KafkaStream

L'application KafkaStream fournie travaille à partir du topic **avro-position** et effectue plusieurs transformations en interne :

- Arrondi les valeurs de latitude et de longitude à une valeur entière
- Transforme le flux en KTable
- Groupe par la position, plutôt que l'id du coursier
- Effectue une agrégation pour maintenir à jour une liste de coursiers pour un position donnée.
Obtient une table
- Déverse le changelog de cette table dans le topic **coursiersParPosition**

Pour démarrer l'application :

```
java -jar streams.jar
```

S'assurer que l'application ne plante pas.

Au bout d'un certain temps le topic de sortie devrait apparaître avec les dernières positions des coursiers.

Relancer une production sur avro et visualiser les mises à jour sur la table.

```
java -jar target/producer-avro-0.0.1-SNAPSHOT-jar-with-dependencies.jar 100 100000 1000  
2
```

8.2 kSQLDb

Objectifs

- Découvrir l'interface CLI de ksqlDB
- Créer des streams et des tables à partir du topic **position** produit aux ateliers précédents
- Écrire des requêtes dérivées pour filtrer et agréger les positions des coursiers
- Comprendre la différence entre Push Query (temps réel) et Pull Query (snapshot)

Contexte

Dans cet atelier, ksqlDB consomme directement le topic **position** produit par JMeter (atelier 2). Aucun code Java n'est nécessaire — tout se fait en SQL depuis la CLI ou via l'API REST.

8.2.1 Démarrage de la stack

Un fichier **docker-compose-ksqldb.yml** vous est fourni. Il étend la stack existante avec :

- **ksqlDB Server** (port 8088)
- **ksqlDB CLI** (client interactif)

```
docker compose -f docker-compose-ksqldb.yml up -d
```

Vérifier que le serveur est opérationnel :

```
curl http://localhost:8088/info
```

8.2.1 Getting Started

Connexion à la CLI

```
docker exec -it ksqldb-cli ksql http://ksqldb-server:8088
```

Explorer l'environnement :

```
SHOW TOPICS;  
SHOW STREAMS;  
SHOW TABLES;
```

Inspecter le topic `position` pour voir les messages existants :

```
PRINT 'position' FROM BEGINNING LIMIT 5;
```

Observer la structure d'un message : `coursierId`, `latitude`, `longitude`, `timestamp`.

Créer le stream de base

Créer un stream `positions` mappé sur le topic `position` :

```
CREATE STREAM positions (  
  coursierId VARCHAR,  
  latitude DOUBLE,  
  longitude DOUBLE  
) WITH (  
  kafka_topic = 'position',  
  value_format = 'JSON'  
);
```

Vérifier la création :

```
DESCRIBE positions;
```

Exécuter une première push query pour voir les messages en temps réel. **Dans un autre terminal**, démarrer la production de message dans le topic `position`

```
SELECT coursierId, latitude, longitude  
FROM positions  
EMIT CHANGES  
LIMIT 10;
```

Observer les messages défiler. Appuyer sur `Ctrl+C` pour arrêter.

8.2.2 Transformations et filtres

Stream dérivé — positions dans Paris

Créer un stream ne contenant que les positions dans la zone parisienne (latitude entre 48.8 et 48.95, longitude entre 2.2 et 2.5) :

```
CREATE STREAM positions_paris AS  
  SELECT coursierId,  
    latitude,  
    longitude  
  FROM positions  
  WHERE latitude BETWEEN 48.80 AND 48.95  
    AND longitude BETWEEN 2.20 AND 2.50  
EMIT CHANGES;
```

Vérifier que le topic correspondant a été créé :

SHOW TOPICS;

Constater que ksqlDB a créé automatiquement le topic POSITIONS_PARIS.

Exécuter une push query sur ce stream filtré :

```
SELECT * FROM positions_paris EMIT CHANGES LIMIT 5;
```

Table — dernière position connue par coursier

Créer une table qui maintient en temps réel la **dernière position connue** de chaque coursier :

```
CREATE TABLE last_position AS
  SELECT courierId,
         LATEST_BY_OFFSET(latitude) AS last_lat,
         LATEST_BY_OFFSET(longitude) AS last_lon,
         COUNT(*) AS nb_messages
  FROM positions
  GROUP BY courierId
  EMIT CHANGES;
```

La table est mise à jour à chaque nouveau message reçu pour un coursier donné.

Agrégation par zone — coursiers par secteur

Arrondir les coordonnées à 1 décimale pour créer des secteurs géographiques, et compter combien de coursiers différents ont été vus dans chaque secteur :

```
CREATE TABLE positions_par_secteur AS
  SELECT CAST(ROUND(last_lat, 1) AS VARCHAR) + '_' +
         CAST(ROUND(last_lon, 1) AS VARCHAR) AS secteur,
         COUNT(*) AS nb_coursiers,
         COLLECT_LIST(courierId) AS coursiers
  FROM last_position
  GROUP BY CAST(ROUND(last_lat, 1) AS VARCHAR) + '_' +
           CAST(ROUND(last_lon, 1) AS VARCHAR)
  EMIT CHANGES;
```

Faire une sélection sur la table

8.2.3 Requêtes

Push Query — suivi en temps réel

Démarrer la production de messages

Dans la CLI, suivre en temps réel les mises à jour de la table last_position :

```
SELECT courierId, last_lat, last_lon, nb_messages
FROM last_position
EMIT CHANGES;
```

Observer les lignes se mettre à jour au fil des messages produits.

Filtrer sur un seul coursier (remplacer `courier-0` par un ID visible dans les logs) :

```
SELECT *
FROM last_position
WHERE courierId = 'courier-0'
EMIT CHANGES;
```

Pull Query — snapshot à l'instant T

Une fois JMeter arrêté, interroger la table pour obtenir l'état courant :

-- Dernière position de tous les coursiers

```
SELECT courierId, last_lat, last_lon, nb_messages
FROM last_position;
```

-- Coursier ayant envoyé le plus de messages

```
SELECT courierId, nb_messages
FROM last_position
WHERE nb_messages > 50;
```

Pull Query vs Push Query : la pull query retourne un snapshot et se termine immédiatement. La push query reste ouverte et pousse les mises à jour en continu.

Push Query avec fenêtre temporelle

Compter les messages par coursier sur des fenêtres glissantes de 1 minute :

```
SELECT courierId,
       WINDOWSTART AS debut,
       WINDOWEND   AS fin,
       COUNT(*)    AS nb
FROM positions
WINDOW TUMBLING (SIZE 1 MINUTE)
GROUP BY courierId
EMIT CHANGES;
```

Relancer la production et observer les compteurs se réinitialiser à chaque nouvelle fenêtre.

8.2.4 Pour aller plus loin

Requête de proximité avec GEO_DISTANCE

Créer une table des coursiers proches de la Place de la République (48.8672, 2.3628) :

```
CREATE TABLE coursiers_proche_republique AS
SELECT courierId,
       last_lat,
       last_lon,
       ROUND(GEO_DISTANCE(last_lat, last_lon, 48.8672, 2.3628), 2) AS distance_km
FROM last_position
WHERE GEO_DISTANCE(last_lat, last_lon, 48.8672, 2.3628) <= 5.0
EMIT CHANGES;
```

Pull query pour voir les coursiers actuellement proches :

```
SELECT * FROM coursiers_proche_republique;
```

Explorer les objets créés

```
DESCRIBE EXTENDED last_position;
SHOW QUERIES;
EXPLAIN <query_id>;
```

DESCRIBE EXTENDED affiche les statistiques d'exécution de la requête sous-jacente.

SHOW QUERIES liste toutes les requêtes persistantes en cours d'exécution.

Nettoyage

```
DROP TABLE coursiers_proche_republique DELETE TOPIC;
DROP TABLE positions_par_secteur      DELETE TOPIC;
```

```
DROP TABLE last_position  
DROP STREAM positions_paris  
DROP STREAM positions;
```

```
DELETE TOPIC;  
DELETE TOPIC;
```


Atelier 9: Sécurité

9.1 Séparation des échanges réseaux

Créer 3 listeners plain text dénommé BROKER, CONTROLLER et EXTERNAL en leur affectant des ports différents

Pour l'instant utiliser des communications en clair pour les 3

Avec un programme précédent utiliser le listener EXTERNAL. Par exemple :

```
java -jar SpringProducer.jar --spring.kafka.producer.bootstrap-servers=localhost:9094
```

et

```
java -jar SpringConsumer.jar --spring.kafka.consumer.bootstrap-servers=localhost:9094
```

9.2 Mise en place de SSL pour crypter les données

Récupérer le répertoire ssl, il contient des certificats pour localhost valable jusqu'en 2033

9.2.1 SSL appliqué aux communications inter-broker

Arrêter les processus qui se connecter au listener PLAINTEXT

Configurer ensuite les fichiers *server.properties*

- Ajouter un listener **BROKER** et l'utiliser pour la communication inter-broker

Configurer le listener BROKER (port, etc..) et les propriétés SSL suivante dans *server.properties*

```
ssl.keystore.location=<path-to>//server.keystore.jks
ssl.keystore.password=secret
ssl.key.password=secret
ssl.truststore.location=<path-to>/server.truststore.jks
ssl.truststore.password=secret
ssl.endpoint.identification.algorithm=
ssl.client.auth=requested
```

Démarrer le cluster kafka et vérifier son bon démarrages

Vérifier l'affichage du certificat avec :

```
openssl s_client -debug -connect localhost:9092 -tls1_3
```

9.2.2 SSL appliqué aux communications externes

Utiliser également le protocole SSL pour le listener EXTERNAL, il suffit de modifier la propriété ***listener.security.protocol.map***

Faire un démarrage du cluster

Mettre au point un fichier ***client-ssl.properties*** avec :

```
security.protocol=SSL
ssl.truststore.location=<path-to>/kafka.truststore.jks
ssl.truststore.password=secret
```

Vérifier la connexion cliente avec les outils Kafka

```
$KAFKA_HOME/bin/kafka-console-producer.sh --bootstrap-server
localhost:9094 --topic ssl --command-config client-ssl.properties
```

```
$KAFKA_HOME/bin/kafka-console-consumer.sh --bootstrap-server
localhost:9094 --topic ssl --command .config client-ssl.properties
--from-beginning
```

Ou en utilisant un des programmes clients précédents

9.3 Authentication via SASL/oAuth2

9.3.1 Démarrage Keycloak

Le docker compose fourni démarre un serveur Keycloak ou est défini un realm kafka comprenant :

- Un client kafka-broker (mdp broker-secret) destiné à être utilisé par les brokers et par les clients d'administration (redpanda et schema-registry)
- Un client kafka-producer-client (mdp producer-secret) destiné à un client producteur
- Un client kafka-consumer-client (mdp consumer-secret) destiné à un client producteur

Démarrer via :

```
docker compose -f docker-compose-keycloak.yml up -d
```

Accéder à l'interface d'administration <http://localhost:9090> admin/admin et visualiser le realm

9.3.2 Démarrage cluster

Ajouter dans la configuration des brokers la configuration oAuth :

```
##### SASL Global
#####
```

```
sasl.mechanism.inter.broker.protocol=OAUTHBEARER
sasl.enabled.mechanisms=OAUTHBEARER
```

```
##### OAUTHBEARER endpoints (global)
#####
```

```
sasl.oauthbearer.token.endpoint.url=http://localhost:9090/realms/kafka/protocol/openid-
connect/token
```

```
sasl.oauthbearer.jwks.endpoint.url=http://localhost:9090/realms/kafka/protocol/openid-connect/certs
sasl.oauthbearer.expected.issuer=http://localhost:9090/realms/kafka
sasl.oauthbearer.expected.audience=kafka-broker
```

```
##### BROKER listener (inter-broker
SASL/OAUTHBEARER) #####
listener.name.broker.oauthbearer.sasl.jaas.config=org.apache.kafka.common.security.oauthbearer.OAuthBearerLoginModule required;
listener.name.broker.oauthbearer.sasl.login.callback.handler.class=org.apache.kafka.common.security.oauthbearer.OAuthBearerLoginCallbackHandler
listener.name.broker.oauthbearer.sasl.server.callback.handler.class=org.apache.kafka.common.security.oauthbearer.OAuthBearerValidatorCallbackHandler
```

```
# Credentials inter-broker (Kafka 4.x — remplace les options JAAS dépréciées)
listener.name.broker.sasl.oauthbearer.client.credentials.client.id=kafka-broker
listener.name.broker.sasl.oauthbearer.client.credentials.client.secret=broker-secret
listener.name.broker.sasl.oauthbearer.scope=openid
```

```
##### EXTERNAL listener (clients
SASL_SSL/OAUTHBEARER) #####
listener.name.external.oauthbearer.sasl.jaas.config=org.apache.kafka.common.security.oauthbearer.OAuthBearerLoginModule required;
listener.name.external.oauthbearer.sasl.server.callback.handler.class=org.apache.kafka.common.security.oauthbearer.OAuthBearerValidatorCallbackHandler
```

Modifier le script de démarrage afin qu'il autorise la connexion au serveur Keycloak :

```
export KAFKA_OPTS="-Dorg.apache.kafka.sasl.oauthbearer.allowed.urls=http://localhost:9090/realms/kafka/protocol/openid-connect/certs,http://localhost:9090/realms/kafka/protocol/openid-connect/token"
```

Redémarrer le cluster et vérifier son bon démarrage

9.3.2 Démarrage outils connexes

Visualiser le fichier docker-compose-kafka-tools.yml et regarder la configuration.

Démarre la stack via :

```
docker compose -f
```

9.3.2 Authentification client

Configurer le listener EXTERNAL pour qu'il utilise également SASL_SSL comme protocole

Mettre à jour le fichier **client-ssl.properties** comme suit :

```
security.protocol=SASL_SSL
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required \
  username="alice" \
  password="alice-secret";
```

Tester avec :

```
$KAFKA_DIST/bin/kafka-console-producer.sh --bootstrap-server  
localhost:9094 --topic ssl --producer.config client-ssl.properties
```

9.4 ACL

9.4.1 Activation des ACLs

Activer les ACL avec la classe ***org.apache.kafka.metadata.authorizer.StandardAuthorizer***

```
authorizer.class.name=org.apache.kafka.metadata.authorizer.StandardAuthorizer
```

Définir les super users et la règle

```
allow.everyone.if.no.acl.found=true
```

```
super.users=User:admin
```

Modifier le protocole du listener CONTROLLER à SASL_SSL

Indiquer la propriété

```
sasl.mechanism.controller.protocol=PLAIN
```

Ajouter une section dans le fichier JAAS :

```
controller.KafkaServer {  
    org.apache.kafka.common.security.plain.PlainLoginModule required  
    username="admin"  
    password="admin-secret"  
    user_admin="admin-secret";  
};
```

Démarrer le cluster

9.4.2 Définition ACLs

Définir une ACL via Redpanda Console ou via `$KAFKA_DIST/bin/kafka-acls.sh` interdisant à l'utilisateur ***alice*** d'écrire sur les topics.

Tester le refus d'envoi de message avec :

```
$KAFKA_DIST/bin/kafka-console-producer.sh --bootstrap-server  
localhost:9094 --topic ssl --producer.config client-ssl.properties
```

9.5 Quota

Définition d'un quota pour l'utilisateur *alice*

```
$KAFKA_DIST/bin/kafka-configs.sh --bootstrap-server localhost:9094  
--alter --add-config  
'producer_byte_rate=1024,consumer_byte_rate=1024' --entity-type  
users --entity-name alice --command-config client-admin-  
ssl.properties
```

Relancer SpringProducer

```
$KAFKA_DIST/bin/kafka-configs.sh --bootstrap-server localhost:9094  
--alter --delete-config producer_byte_rate --entity-type users --  
entity-name alice --command-config client-admin-ssl.properties
```

```
$KAFKA_DIST/bin/kafka-configs.sh --bootstrap-server localhost:9094  
--alter --delete-config consumer_byte_rate --entity-type users --  
entity-name alice --command-config client-admin-ssl.properties
```

Atelier 10 : Exploitation

Exécuter les producteurs et les consommateurs pendant les opérations d'administration

10.1 Rétention et segments

Visualiser les segments et apprécier la taille

Pour le topic *position* modifier le *segment.bytes* à 512ko

Diminuer le *retention.bytes* afin de voir des segments disparaître

10.2 Extension du cluster et réassignement des partitions

Modifier le nombre de partitions de position à 8

Vérifier avec

```
./kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic position
```

Ajouter un nouveau broker dans le cluster. (Ne pas l'inclure dans les contrôleurs possible)

Option Docker : un docker-compose est fourni

Réexécuter la commande

```
./kafka-topics.sh --bootstrap-server localhost:909 --describe --topic position
```

Effectuer une réaffectation des partitions en 3 étapes

10.3 Rolling restart

Sous charge, effectuer un redémarrage d'un broker.

Vérifier l'état des ISR

10.4 Bascule et reprise depuis le cluster B

Objectifs

- Simuler une indisponibilité du cluster A
- Faire reprendre un consumer côté B sans rejouer toute l'historique
- Mesurer le RPO observé et constater les limites

10.4.1. Préparer un consumer group côté A

Côté A, lancer un consumer dans un groupe nommé position-app qui consomme une partie des messages puis s'arrête :

```
docker exec -it kafka-0 /opt/kafka/bin/kafka-console-consumer.sh \
```

```
--bootstrap-server kafka-0:9092 --topic position \  
--group position-app --from-beginning --max-messages 10
```

Vérifier l'offset committé :

```
docker exec -it kafka-0 /opt/kafka/bin/kafka-consumer-groups \  
--bootstrap-server kafka-0:9092 --describe --group position-app
```

Attendre ~10 secondes que le MirrorCheckpointConnector propage les offsets vers B (intervalle configuré à 5s en 7.4).

10.4.2. Inspecter les checkpoints côté B

Le connecteur Checkpoint écrit dans un topic dédié côté B. Le visualiser :

```
docker exec -it kafka-b /opt/kafka/bin/kafka-console-consumer.sh \  
--bootstrap-server kafka-b:9092 \  
--topic A.checkpoints.internal --from-beginning \  
--formatter org.apache.kafka.connect.mirror.formatters.CheckpointFormatter
```

On y voit pour chaque partition : l'offset upstream (côté A) et l'offset downstream traduit (côté B). C'est la base de la reprise.

10.4.3. Simuler la panne de A

```
docker stop kafka-0 kafka-1 kafka-2
```

Le cluster A est injoignable. MirrorMaker s'arrête de répliquer (visible dans les logs Connect), mais B contient déjà les données.

10.4.4. Faire reprendre le consumer côté B

MirrorMaker 2 ne pousse pas automatiquement les offsets dans `__consumer_offsets` côté B — il faut les *appliquer* au groupe avant de redémarrer le consumer. L'outil `RemoteClusterUtils` est exposé via la classe utilitaire `kafka-mirror-checkpoint.sh` (Kafka 3.5+). Sur des versions antérieures, on passe par un petit script Java/Python ou par `kafka-consumer-groups.sh --reset-offsets --to-offset` en lisant manuellement le topic checkpoints.

Variante simple pour l'atelier — application manuelle :

```
# Lire le dernier checkpoint pour position-app  
# (offset downstream côté B pour chaque partition)  
  
# Appliquer cet offset au groupe côté B  
docker exec -it kafka-b /opt/kafka/bin/kafka-consumer-groups.sh \  
--bootstrap-server kafka-b:9092 \  
--group position-app --topic A.position \  
--reset-offsets --to-offset <OFFSET_DOWNSTREAM> --execute
```

Puis relancer le consumer côté B :

```
docker exec -it kafka-b /opt/kafka/bin/kafka-console-consumer.sh \  
--bootstrap-server kafka-b:9092 --topic A.position \  
--group position-app
```

Constat clé : il reprend à la suite de ce qu'il avait déjà traité côté A, **pas depuis le début**.

10.4.5. Mesurer le RPO

Comparer :

- Le dernier offset produit côté A avant l'arrêt (visible dans les logs ou via `kafka-run-class kafka.tools.GetOffsetShell` avant `docker stop`)
- Le dernier offset présent côté B sur `A.position`

L'écart = **RPO observé** (messages perdus si A est définitivement perdu).

Discussion à animer :

- Pourquoi cet écart existe-t-il ? → réplication asynchrone
- Pourquoi l'offset traduit n'est-il pas exact au message près ? → la traduction se fait par échantillonnage périodique via `mm2-offset-syncs`
- Comment réduire le RPO ? → diminuer `emit.checkpoints.interval.seconds`, augmenter `tasks.max`, mais aucun moyen d'atteindre RPO=0 avec MM2

10.4.6. Failback

Redémarrer A (`docker start kafka-0 kafka-1 kafka-2`) et discuter :

- Faut-il rebasculer ? Quand ?
- Comment éviter la double-écriture pendant la transition ?
- Intérêt d'une réplication bidirectionnelle $A \leftrightarrow B$ (et son piège : éviter les boucles avec `IdentityReplicationPolicy` ou les tags d'origine).

Atelier 11. Monitoring avec Prometheus/Grafana

Dans un premier temps, démarré une **JConsole** et visualiser les Mbeans des brokers, consommateurs et producteurs

```
$JAVA_HOME/bin/jconsole &
```

11.1 Exporter les informations JMX au format Prometheus

Dans un répertoire de travail **monitoring**

```
wget  
https://repo1.maven.org/maven2/io/prometheus/jmx/jmx\_prometheus\_javaagent/0.20.0/jmx\_prometheus\_javaagent-0.20.0.jar
```

```
wget  
https://raw.githubusercontent.com/confluentinc/jmx-monitoring-stacks/main/shared-assets/jmx-exporter/kafka\_broker.yml
```

Modifier le script de démarrage du cluster afin de positionner l'agent Prometheus :

```
KAFKA_OPTS="$KAFKA_OPTS -javaagent:$PWD/jmx_prometheus_javaagent-0.20.0.jar=7071:$PWD/kafka_broker.yml" \
```

Attention, Modifier le port pour chaque broker

Redémarrer le cluster et vérifier <http://localhost:7071/metrics> (Peut prendre du temps avant de s'afficher)

9.3 Mise en place Prometheus Grafana

Dans le répertoire grafana des données du TP, lancer la stack via :

```
docker compose up -d
```

Se connecter sur **<http://localhost:9090>**

Vérifier les cibles de prometheus

Accéder à <http://localhost:3000> avec **admin/admin**

Vérifier la datasource Prometheus

Importer le tableau de bord fourni (Provient de <https://github.com/confluentinc/jmx-monitoring->

[stacks/tree/6.0.1-post/jmxexporter-prometheus-grafana/assets/prometheus/grafana/provisioning/dashboards\)](#)

Les métriques des brokers doivent s'afficher

A. Annexes

A.1. Installation Ensemble Zookeeper

Récupérer une distribution de Zookeeper.

Créer un répertoire (dans la suite nommé *ZK_ENSEMBLE*) pour stocker les fichiers de configuration des instance

Y créer 3 sous-répertoire 1,2 et 3

Dans chacun des répertoire créer un sous-répertoire **data** y créer un fichier nommé **myid** contenant l'identifiant de l'instance (1,2 et 3)

Copier dans chacun des répertoires le répertoire **conf** de la distribution

Dans chacun des répertoire éditer le fichier **zoo.cfg** , en particulier

- *dataDir*
- Les ports TCP utilisés

Se mettre au point un script sh permettant de démarrer les 3 instances

```
$ZK_HOME/bin/zkServer.sh --config $ZK_ENSEMBLECONF/1/conf start &&  
$ZK_HOME/bin/zkServer.sh --config $ZK_ENSEMBLECONF/2/conf start  
&&  
$ZK_HOME/bin/zkServer.sh --config $ZK_ENSEMBLECONF/3/conf start
```

Données partagées :

Connecter à l'instance 1 via :

```
./zookeeper-1/bin/zkCli.sh -server 127.0.0.1:2181
```

Créer une donnée et enregistrer un *watcher* :

```
create /dp hello
```

```
get -w /dp
```

Dans un autre terminal, se connecter à l'instance 2 et mettre à jour la donnée partagée

```
./zookeeper-2/bin/zkCli.sh -server 127.0.0.1:2182  
set /dp world
```

Vous devez observer une notification dans le terminal 1

Election et quorum

Afficher les rôles des serveurs avec la commande :

```
echo stat | nc localhost 2181 | grep Mode && echo stat | nc  
localhost 2182 | grep Mode && echo stat | nc localhost 2183 | grep  
Mode
```

Stopper le serveur leader et observer la réélection

Stopper encore un serveur et interroger le serveur restant. L'ensemble n'ayant plus de quorum ne doit pas répondre

Redémarrer un serveur et observer la remise en service de le l'ensemble